Data Processing: Stony Creek, Tivoli North, and Field Station

Ilianna Ditren, Joy Terentiev, Natalie Novoseltseva

This document contains the processing steps for downloading the main datasets. Initially, water quality data collected from Stony Creek and weather data collected from Field Station were explored individually. In the following steps, we first download each dataset, place it in a new folder, and then we combine the two datasets in order to perform multivariate exploratory data analysis.

Project Set Up

Downloading libraries

```
library(readr)
library(tidyverse)
library(dplyr)
library(lubridate)
library(janitor)
library(stringr)
library(reshape2)
```

Data and Cleaning

To obtain raw water quality and weather data, use the Centralized Data Management Office (CDMO) website: https://cdmo.baruch.sc.edu/

1. Navigate to the CDMO website: https://cdmo.baruch.sc.edu/.
2. Click on the **Get Data** tab at the top of the page.
3. Select **Advanced Query System** and click **Launch**.
4. Locate the **Zip Downloads** option and click **Choose ZIP Files**.

5. From the list of National Estuarine Research Reserves, select **Hudson River, NY**.
6. Select the appropriate dataset:
    - For **weather data** from the field station, select **hudfsmet-p**.
    - For **Stony Creek water quality data**, select **hudscwq-p**.
    - For **Tivoli North water quality data**, select **hudtnwq-p**.

    Each dataset must be requested separately, as the website generates one download link per request, and the analysis code is structured to process each dataset independently.
7. After selecting the desired site, click **Submit location and proceed to next step.**
8. Select the desired range of years and click **Get files**.
    - For the DA Capstone Spring 2026 project, requests were made for all available years.
9. Complete the required information requested by the website and submit the request.
10. Shortly after submission, an email will be sent to the provided email address containing a download link to the data files.
11. Copy and paste this link into the corresponding section of the code for the selected site.
12. Run the code. After execution, all files will be downloaded into a subfolder named after the site within your data folder.

Note: Download link expires after certain amount of time. Ensure to download the data soon after you receive the link.

Stony Creek Water Quality (2002-2021)

Data Download

```
#creates a folder for the new data
dir.create("Spring2026/data/stony_creek", recursive = TRUE, showWarnings = FALSE)

## downloading Stony Creek data file from the web

#creating a temporary zip file
zip_sc <- tempfile(fileext = ".zip")
```

```
stony_creek_raw <- "http://cdmo.baruch.sc.edu/aqs/output/60521.zip"

download.file(stony_creek_raw, zip_sc, mode = "wb")

#unzipping data from zipfile and placing it in a folder
unzip(zip_sc, exdir = "Spring2026/data/stony_creek")
```

```
#read in the files for all the years as a one data file using regex (regular expressions)
files <- list.files(
  "data/stony_creek",
  pattern = "^hudscwq[0-9]{4}\\.csv$", # ^ -starts a string, hudscwq - the exact name in the
  full.names = TRUE
)

sc_02_21_raw <- do.call(
  rbind,
  lapply(files, function(f) {
    df <- read.csv(f, fileEncoding = "Latin1")
    df$year <- as.integer(gsub(".*([0-9]{4})\\.csv$", "\\1", f))
    df
  })
)
```

Data Cleaning

```
# Removing empty columns and other irrelevant columns
t <- sc_02_21_raw |>
  remove_empty("cols")|>
  select(-StationCode,
         -isSWMP, #historical and provisionalPlus columns indicate the status of QAQC
         -Historical,
         -ProvisionalPlus
  )

#creating a vector with all the flags
flags <- c("F_Record",
           "F_Temp",
           "F_SpCond",
           "F_Sal",
           "F_DO_Pct",
```

```
            "F_Depth",
            "F_cDepth",
            "F_Level",
            "F_pH",
            "F_Turb",
            "F_ChlFluor",
            "F_DO_mgl")

t <- t |>
  mutate(Temp = if_else(
      if_any(all_of(flags), ~ str_detect(.x, "<-3>|<-2>|<1>")), NA_real_, Temp))
```

```
stony_creek <- stony_creek |>
  mutate(Temp = if_else(str_detect(F_Temp, "<-3>|<-2>|<1>"), NA_real_, Temp),
         SpCond = if_else(str_detect(F_SpCond, "<-3>|<-2>|<1>"), NA_real_, SpCond),
         Sal = if_else(str_detect(F_Sal, "<-3>|<-2>|<1>"), NA_real_, Sal),
         DO_Pct = if_else(str_detect(F_DO_Pct, "<-3>|<-2>|<1>"), NA_real_, DO_Pct),
         Depth = if_else(str_detect(F_Depth, "<-3>|<-2>|<1>"), NA_real_, Depth),
         cDepth = if_else(str_detect(F_cDepth, "<-3>|<-2>|<1>"), NA_real_, cDepth),
         pH = if_else(str_detect(F_pH, "<-3>|<-2>|<1>"), NA_real_, pH),
         Turb = if_else(str_detect(F_Turb, "<-3>|<-2>|<1>"), NA_real_, Turb),
         ChlFluor = if_else(str_detect(F_ChlFluor, "<-3>|<-2>|<1>"), NA_real_, ChlFluor),
         DO_mgl = if_else(str_detect(DO_mgl, "<-3>|<-2>|<1>"), NA_real_, DO_mgl)
  )

#stony_creek <- stony_creek |>
#  mutate(
#    Temp = if_else(str_detect(F_Temp, "<-3>|<-2>") | (str_detect(F_Temp, "<1>") & year != 20
#    SpCond = if_else(str_detect(F_SpCond, "<-3>|<-2>") | (str_detect(F_SpCond, "<1>") & yea
#    Sal = if_else(str_detect(F_Sal, "<-3>|<-2>") | (str_detect(F_Sal, "<1>") & year != 2019)
#    DO_Pct = if_else(str_detect(F_DO_Pct, "<-3>|<-2>") | (str_detect(F_DO_Pct, "<1>") & yea
#    Depth = if_else(str_detect(F_Depth, "<-3>|<-2>") | (str_detect(F_Depth, "<1>") & year !=
#    cDepth = if_else(str_detect(F_cDepth, "<-3>|<-2>") | (str_detect(F_cDepth, "<1>") & yea
#    pH = if_else(str_detect(F_pH, "<-3>|<-2>") | (str_detect(F_pH, "<1>") & year != 2019), I
#    Turb = if_else(str_detect(F_Turb, "<-3>|<-2>") | (str_detect(F_Turb, "<1>") & year != 20
#    ChlFluor = if_else(str_detect(F_ChlFluor, "<-3>|<-2>") | (str_detect(F_ChlFluor, "<1>")
#    DO_mgl = if_else(str_detect(F_DO_mgl, "<-3>|<-2>") | (str_detect(F_DO_mgl, "<1>") & yea
  )

#removing flag columns
stony_creek <- stony_creek %>% select(-all_of(flags))
```

Cleaning and creating new date and time columns

```
#separate the date and time column
stony_creek <- stony_creek %>%
  mutate(
    DateTimeStamp = as.POSIXct(DateTimeStamp, format = "%m/%d/%Y %H:%M"), #converts the varia
    date = as.Date(DateTimeStamp),
    time = format(DateTimeStamp, "%H:%M:%S")
  )

#create a month column
stony_creek <- stony_creek %>%
  mutate(
    month = month(DateTimeStamp)
  )

#create a season column
stony_creek <- stony_creek %>%
  mutate(
    season = case_when(
      month %in% c(12, 1, 2) ~ "Winter",
      month %in% 3:5         ~ "Spring",
      month %in% 6:8         ~ "Summer",
      month %in% 9:11        ~ "Fall"
    )
  )
```

```
#the code to write out the dataset
write_csv(stony_creek, "Spring2026/data/stony_creek.csv")

#uncomment the code to write out the new/updated version of the dataset. It might be better t
```

Tivoli North Water Quality (1996 - 2025)

Data Download

```
#creates a folder for the new data
dir.create("data/tivoli_north", recursive = TRUE, showWarnings = FALSE)
```

```r
#creating a temporary zip file
zip_tn <- tempfile(fileext = ".zip")

tiv_north_raw <- "http://cdmo.baruch.sc.edu/aqs/output/444944.zip"

download.file(tiv_north_raw, zip_tn, mode = "wb")

#unzipping data from and placing it the created folder
unzip(zip_tn, exdir = "data/tivoli_north")
```

```r
#read in the files for all the years as a one data file
files <- list.files(
  "data/tivoli_north",
  pattern = "^hudtnwq[0-9]{4}\\.csv$", # ^ -starts a string, hudtnwq - the exact name in the
  full.names = TRUE
)

tn_96_25 <- do.call(
  rbind,
  lapply(files, function(f) {
    df <- read.csv(f, fileEncoding = "Latin1")
    df$year <- as.integer(gsub(".*([0-9]{4})\\.csv$", "\\1", f))
    df
  })
)
```

Field Station Meteorological Data (2001-2026)

Data Download

```r
#creates a data folder
dir.create("Spring2026/data/field_station_weather", recursive = TRUE, showWarnings = FALSE)

#using tempfile() to create a tempoary storage
zip <- tempfile(fileext = ".zip")

#downlod the file from the url
download.file("http://cdmo.baruch.sc.edu/aqs/output/854334.zip", zip, mode = "wb")
```

```r
#unzipping data from and placing it the created folder
unzip(zip, exdir = "Spring2026/data/field_station_weather")

#read in the files for all the years as a one data file hudfsmet-p
files <- list.files(
  "Spring2026/data/field_station_weather",
  pattern = "^hudfsmet[0-9]{4}\\.csv$", # ^ -starts a string
                                        # hudfsmet - the exact name in the file (constant)
                                        # any digits from 0-9 might follow
                                        # {4} - we are looking for 4 digits pattern
                                        # \\. - there should be a dot
                                        # csv - it should end with "csv"
                                        # end a string
  full.names = TRUE
)

fs_weather_01_26_raw <- do.call(
  rbind,
  lapply(files, function(f) {
    df <- read.csv(f, fileEncoding = "Latin1")
    df$year <- as.integer(gsub(".*([0-9]{4})\\.csv$", "\\1", f))
    df
  })
)
```

Data Cleaning

```r
# Removing empty columns and other irrelevant columns
fs_weather_01_26 <- fs_weather_01_26_raw |>
  remove_empty("cols")|>
  select(-StationCode,
         -isSWMP, #historical and provisional Plus columns indicate the status of QAQC
         -Historical,
         -ProvisionalPlus,
  )
```

```r
#creating a vector with all the weather flags
flags_weather <- c(
  "F_Record",
  "F_ATemp",
```

```
  "F_RH",
  "F_BP",
  "F_WSpd",
  "F_MaxWSpd",
  "F_Wdir",
  "F_SDWDir",
  "F_TotPAR",
  "F_TotPrcp",
  "F_TotSoRad")

#creating a vector with all the weather flags that are to be removed from the dataset
flagged_data <- "<-3>|<-2>|<1>"

fs_weather_01_26 <- fs_weather_01_26 |>
  mutate(ATemp = if_else(str_detect(F_ATemp, flagged_data), NA_real_, ATemp),
         RH = if_else(str_detect(F_RH, flagged_data), NA_real_, RH),
         BP = if_else(str_detect(F_BP, flagged_data), NA_real_, BP),
         WSpd = if_else(str_detect(F_WSpd, flagged_data), NA_real_, WSpd),
         MaxWSpd = if_else(str_detect(F_MaxWSpd, flagged_data), NA_real_, MaxWSpd),
         Wdir = if_else(str_detect(F_Wdir, flagged_data), NA_real_, Wdir),
         SDWDir = if_else(str_detect(F_SDWDir, flagged_data), NA_real_, SDWDir),
         TotPAR = if_else(str_detect(F_TotPAR, flagged_data), NA_real_, TotPAR),
         TotPrcp = if_else(str_detect(F_TotPrcp, flagged_data), NA_real_, TotPrcp)
  )

#removing flag columns
fs_weather_01_26 <- fs_weather_01_26 |>
  select(-all_of(flags_weather))
```

```
#separate the data and time column
fs_weather_01_26 <- fs_weather_01_26 %>%
  mutate(
    DatetimeStamp = as.POSIXct(DatetimeStamp, format = "%m/%d/%Y %H:%M"), #converts the varia
    date = as.Date(DatetimeStamp),
    time = format(DatetimeStamp, "%H:%M:%S")
  )

#create a month column
fs_weather_01_26 <- fs_weather_01_26 %>%
  mutate(
    month = month(DatetimeStamp)
  )
```

```r
#create a season column
fs_weather_01_26 <- fs_weather_01_26 %>%
  mutate(
    season = case_when(
      month %in% c(12, 1, 2) ~ "Winter",
      month %in% 3:5         ~ "Spring",
      month %in% 6:8         ~ "Summer",
      month %in% 9:11        ~ "Fall"
    )
  )
```

```r
#the code to write out the dataset
write_csv(fs_weather_01_26, "Spring2026/data/fs_weather_01_26.csv")

#uncomment the code to write out the new/updated version of the dataset. It might be better t
```

Joined dataset (Stony Creek Water Quality and Field Station Weather Data)

```r
# reading main datasets
water <- read_csv("data/stony_creek.csv")
weather <- read_csv("data/fs_weather_01_26.csv")
```

Exploring the structure of the data

```r
str(weather)
str(water)

names(weather)
names(water)
```

```r
#rename the column in the water dataset to easier join the datasets
weather <- weather %>%
  rename(DateTimeStamp = DatetimeStamp)
```

Joining the water quality and weather datasets

```
#join water and weather data with inner join
combined <- inner_join(water, weather, by = "DateTimeStamp")
```

```
#get rid of the duplicate columns and
combined <- combined %>%
  select(-year.y, -date.y, -time.y, -month.y, -season.y) %>%
  rename(
    year = year.x,
    date = date.x,
    time = time.x,
    month = month.x,
    season = season.x
  )
```

```
#the code to write out the dataset
write_csv(combined, data/combined_dataset/joined_dataset.csv")
```

The combined dataset contained variables measured every 15-30 minutes. In this section, the data will be summarized to contain daily averages only.

```
# recode the seasons:

combined <-combined %>%
  mutate(
  season_astro = case_when(
    date >= as.Date(paste0(year, "-12-21")) | date < as.Date(paste0(year, "-03-20")) ~ "Winte
    date >= as.Date(paste0(year, "-03-20")) & date < as.Date(paste0(year, "-06-21")) ~ "Spri
    date >= as.Date(paste0(year, "-06-21")) & date < as.Date(paste0(year, "-09-22")) ~ "Summe
    TRUE ~ "Fall"
  )
)

#creating a vector with all numerical variable that need mean computed and extracting a name

raw_combined <- combined |>
  mutate(season_num = case_when(
    season == "Winter" ~ 1,
    season == "Spring" ~ 2,
    season == "Summer" ~ 3,
```

```
    season == "Fall" ~ 4,
  ))

# computing the daily mean values for all variables in the combined_names object
daily_combined <- combined |>
  group_by(date) |>
  summarize(across(
    where(is.numeric), ~ mean(.x, na.rm = TRUE)),.groups = "drop") |>
  mutate(
  season_astro = case_when(
    date >= as.Date(paste0(year, "-12-21")) | date < as.Date(paste0(year, "-03-20")) ~ "Winte
    date >= as.Date(paste0(year, "-03-20")) & date < as.Date(paste0(year, "-06-21")) ~ "Sprin
    date >= as.Date(paste0(year, "-06-21")) & date < as.Date(paste0(year, "-09-22")) ~ "Summe
    TRUE ~ "Fall"
  )
)

# adding the season column that got lost.

#writing new csv
#write_csv(daily_combined, "Spring2026/data/combined_dataset/daily_combined.csv")
```

```
# calculate weekly averages:
combined <- combined %>% #create a new dataset
  mutate(week = floor_date(date, unit = "week")) %>% #create a new "week" column
  group_by(week) %>% #group by week
  summarise(
    across(
      where(is.numeric), #apply only numeric columns
      ~ mean(.x, na.rm = TRUE) #calculate the mean
    ),
    .groups = "drop"
  )
```